DAY 76 | BEAN VALIDATION | JAKARTA VALIDATION 3.1 | SPRING BOOT 4.1

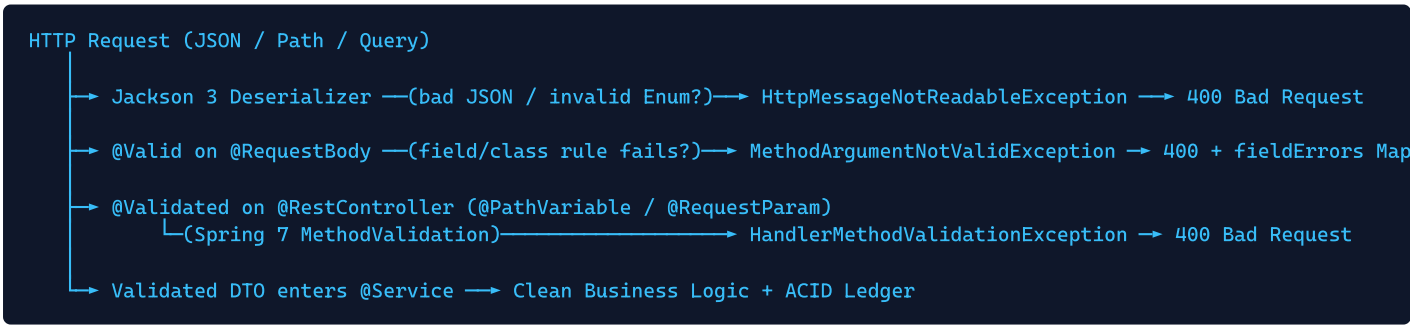
Bean Validation & API Input Hardening

Study Notes & Planned Implementation Blueprint: Declarative DTO & Parameter Validation, Custom Banking Constraints, Cross-Field Rules, Monetary Precision Guards, and Structured Global Error Handling.

PROJECT	ENGINE	SPRING STACK	TARGET LAYER
Enterprise Digital Banking	**Hibernate Validator 9.1**	**Boot 4.1.0 / Web 7.0.8**	**DTO + Controller + Advice**

1. Core Concepts: How Bean Validation Works in Spring Boot 4

In an enterprise banking API, every byte crossing the HTTP boundary is untrusted input. Relying only on database `CHECK` constraints (Day 70) or manual `if/else` checks in service methods wastes database connections, thread-pool slots, and Kafka I/O when malformed inputs can be rejected in microseconds at the Web/Controller boundary.



`@Valid` (Jakarta Standard)

- Triggers validation on `@RequestBody` and `@ModelAttribute` objects.
- Cascades validation recursively into nested objects/lists when placed on fields.
- Throws `MethodArgumentNotValidException` in Spring MVC.
- Does **NOT** support validation groups or method-level primitives on its own.

`@Validated` (Spring Framework)

- Placed on the `@RestController` class to enable proxy/method validation for `@PathVariable` and `@RequestParam` (e.g., `@Positive Long accountId`).
- Supports **Validation Groups** (e.g., `OnCreate.class` vs `OnUpdate.class`).
- In Spring Framework 6.1+ / 7.0 (Boot 4.1), triggers `HandlerMethodValidationException` natively inside `RequestMappingHandlerAdapter`.

Essential Constraint Distinctions (Interview Favorite)

Annotation	Rejects `null`?	Rejects `""`?	Rejects `" "`?	Banking Usage Rule
`@NotNull`	YES	NO	NO	Enums (`AccountType`), IDs (`Long`), `BigDecimal` , `LocalDate` .
`@NotEmpty`	YES	YES	NO	Non-empty collections / arrays (`List<Long>`).
`@NotBlank`	YES	YES	YES	Strings: `branch` , `email` , `reason` , `panNumber` .
`@Digits(integer=16, fraction=2)`	No (pair w/ `@NotNull`)	N/A	N/A	Matches MySQL `DECIMAL(18,2)` ; stops sub-paise inputs like `100.1234` !

2. Real Codebase Gap Audit (What Exists vs. What Day 76 Fixes)

Before writing code, we audited every controller, request DTO, and `GlobalExceptionHandler` already in `org.example.enterprisedigitalbankingsystem` :

Component Audited	Current State in Codebase	Planned Day 76 Upgrade
GlobalExceptionHandler	Handles `MethodArgumentNotValidException` as a flat string; missing handlers for method-param errors and invalid JSON/enums (currently leaks **500 Error**!).	Handle `HandlerMethodValidationException` , `ConstraintViolationException` , and `HttpMessageNotReadableException` cleanly as `400` ; add structured `fieldErrors` map to `ErrorResponse` .
Controllers (Path/Query)	Use `@Valid @RequestBody` , but **none** validate `@PathVariable` IDs (`-1` or `0` hits DB unnecessarily).	Add `@Validated` + `@Positive` / `@NotBlank` on all controller path/query parameters.
Money Fields (BigDecimal)	`DepositRequest` , `WithdrawRequest` , `TransferRequest` , `CreateAccountRequest` , `CreateLoanRequest` check `@DecimalMin` only.	Enforce `@Digits(integer = 16, fraction = 2)` and `@DecimalMax` (e.g., `100000000.00`) aligned with MySQL `DECIMAL(18,2)` .
Customer DOB	`CreateCustomerRequest` / `UpdateCustomerRequest` only have `@NotNull` on `dateOfBirth` (allows future year 2099!).	Add `@Past` + custom `@MinimumAge(18)` banking KYC rule validator.
TransferRequest (XOR Rule)	`destinationAccountId` vs `beneficiaryId` exclusivity and self-transfer checks live inside service code only.	Create class-level `@ValidTransferTarget` constraint so invalid requests fail at Controller level immediately.
Password Confirmation	`RegisterRequest` , `ResetPasswordRequest` , `ChangePassowordRequest` check password match manually in service; typos in validation messages.	Create `@PasswordMatches` class-level validator + complete messages & strong constraints across all auth DTOs.

3. Planned Code: Structured Error Handling & Hardened GlobalExceptionHandler

We keep `ErrorResponse` 100% backward-compatible (keeping its 4-argument constructor) while adding optional `fieldErrors` populated during validation failures and suppressed via `@JsonInclude(NON_NULL)` on non-validation errors.

src/main/java/org/example/enterprisedigitalbankingsystem/exception/ErrorResponse.java MODIFIED

```java
package org.example.enterprisedigitalbankingsystem.exception;

import com.fasterxml.jackson.annotation.JsonInclude;
import java.time.LocalDateTime;
import java.util.Map;

@JsonInclude(JsonInclude.Include.NON_NULL)
public class ErrorResponse {

    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String path;
    private Map<String, String> fieldErrors;

    public ErrorResponse() {}

    // Preserved 4-arg constructor for 100% compatibility with existing handlers & tests
    public ErrorResponse(LocalDateTime timestamp, int status, String error, String path) {
        this(timestamp, status, error, path, null);
    }

    public ErrorResponse(LocalDateTime timestamp, int status, String error,
                         String path, Map<String, String> fieldErrors) {
        this.timestamp = timestamp;
        this.status = status;
        this.error = error;
        this.path = path;
        this.fieldErrors = fieldErrors;
    }

    // Standard getters and setters for all 5 fields ...
}
```

src/main/java/org/example/enterprisedigitalbankingsystem/exception/GlobalExceptionHandler.java MODIFIED

```java
import jakarta.validation.ConstraintViolationException;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.method.annotation.HandlerMethodValidationException;
import java.util.LinkedHashMap;
import java.util.Map;

// 1. @RequestBody DTO field & class-level constraint violations (400)
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidationException(
        MethodArgumentNotValidException ex, HttpServletRequest request) {

    Map<String, String> fieldErrors = new LinkedHashMap<>();
    ex.getBindingResult().getFieldErrors().forEach(fe ->
            fieldErrors.putIfAbsent(fe.getField(), fe.getDefaultMessage()));
    ex.getBindingResult().getGlobalErrors().forEach(ge ->
            fieldErrors.putIfAbsent(ge.getObjectName(), ge.getDefaultMessage()));

    String summary = fieldErrors.entrySet().stream()
            .map(e -> e.getKey() + ": " + e.getValue())
            .collect(Collectors.joining(", "));

    return new ResponseEntity<>(
            new ErrorResponse(LocalDateTime.now(), HttpStatus.BAD_REQUEST.value(),
                    summary, request.getRequestURI(), fieldErrors),
```

```
            HttpStatus.BAD_REQUEST);
}

// 2. Spring Boot 4 / Framework 7 @PathVariable & @RequestParam validation errors (400)
@ExceptionHandler(HandlerMethodValidationException.class)
public ResponseEntity<ErrorResponse> handleMethodValidation(
        HandlerMethodValidationException ex, HttpServletRequest request) {

    Map<String, String> fieldErrors = new LinkedHashMap<>();
    ex.getParameterValidationResults().forEach(result -> {
        String param = result.getMethodParameter().getParameterName();
        result.getResolvableErrors().forEach(err ->
                fieldErrors.putIfAbsent(param != null ? param : "parameter", err.getDefaultMessage()));
    });

    return new ResponseEntity<>(
            new ErrorResponse(LocalDateTime.now(), HttpStatus.BAD_REQUEST.value(),
                    "Validation failed for request parameter(s)", request.getRequestURI(), fieldErrors),
            HttpStatus.BAD_REQUEST);
}

// 3. Direct Bean Validation / Service-layer ConstraintViolationException (400)
@ExceptionHandler(ConstraintViolationException.class)
public ResponseEntity<ErrorResponse> handleConstraintViolation(
        ConstraintViolationException ex, HttpServletRequest request) {

    Map<String, String> fieldErrors = new LinkedHashMap<>();
    ex.getConstraintViolations().forEach(v ->
            fieldErrors.putIfAbsent(v.getPropertyPath().toString(), v.getMessage()));

    return new ResponseEntity<>(
            new ErrorResponse(LocalDateTime.now(), HttpStatus.BAD_REQUEST.value(),
                    "Constraint violation", request.getRequestURI(), fieldErrors),
            HttpStatus.BAD_REQUEST);
}

// 4. Malformed JSON, wrong date format, or invalid Enum constant (e.g. accountType="CRYPTO")
@ExceptionHandler(HttpMessageNotReadableException.class)
public ResponseEntity<ErrorResponse> handleUnreadableMessage(
        HttpMessageNotReadableException ex, HttpServletRequest request) {
    return buildResponse(HttpStatus.BAD_REQUEST,
            "Malformed JSON payload or invalid field value/enum format", request);
}
```

> **Why `HttpMessageNotReadableException` matters:** Previously, sending `{"accountType": "INVALID"}` threw `HttpMessageNotReadableException`, which fell through to `handleGenericException` and returned an alarming **500 Internal Server Error**! Catching it returns a clean **400 Bad Request**.

4. Planned Code: Custom Banking Constraint Annotations

Custom Jakarta Bean Validation annotations eliminate repetitive validation code across controllers and DTOs while expressing real domain rules declaratively.

`src/main/java/ … /common/validation/MinimumAge.java & MinimumAgeValidator.java` NEW

```
package org.example.enterprisedigitalbankingsystem.common.validation;

import jakarta.validation.Constraint;
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import jakarta.validation.Payload;
import java.lang.annotation.*;
import java.time.LocalDate;
import java.time.Period;

@Target({ElementType.FIELD, ElementType.PARAMETER})
```

```java
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Constraint(validatedBy = MinimumAgeValidator.class)
public @interface MinimumAge {
    int value() default 18;
    String message() default "Customer must be at least {value} years old";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class MinimumAgeValidator implements ConstraintValidator<MinimumAge, LocalDate> {
    private int minimumAge;

    @Override
    public void initialize(MinimumAge constraintAnnotation) {
        this.minimumAge = constraintAnnotation.value();
    }

    @Override
    public boolean isValid(LocalDate dob, ConstraintValidatorContext context) {
        if (dob == null) return true; // @NotNull handles nullability independently
        return Period.between(dob, LocalDate.now()).getYears() ≥ minimumAge;
    }
}
```

src/main/java/ ... /transaction/dto/request/TransferRequest.java (Cross-Field Bean Validation) MODIFIED

```java
package org.example.enterprisedigitalbankingsystem.transaction.dto.request;

import jakarta.validation.constraints.*;
import lombok.*;
import java.math.BigDecimal;

@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class TransferRequest {

    @NotNull(message = "Source account id is required")
    @Positive(message = "Source account id must be positive")
    private Long sourceAccountId;

    @Positive(message = "Destination account id must be positive")
    private Long destinationAccountId;

    @Positive(message = "Beneficiary id must be positive")
    private Long beneficiaryId;

    @NotNull(message = "Amount is required")
    @DecimalMin(value = "1.00", message = "Amount must be at least 1.00")
    @DecimalMax(value = "10000000.00", message = "Single transfer limit is 10,000,000.00")
    @Digits(integer = 16, fraction = 2, message = "Amount can have at most 2 decimal places")
    private BigDecimal amount;

    @Size(max = 255, message = "Remarks cannot exceed 255 characters")
    private String remarks;

    @AssertTrue(message = "Provide exactly one of destinationAccountId or beneficiaryId")
    public boolean isTargetExclusive() {
        return (destinationAccountId ≠ null) ^ (beneficiaryId ≠ null);
    }

    @AssertTrue(message = "Source and destination account cannot be the same")
    public boolean isNotSelfTransfer() {
        if (sourceAccountId == null || destinationAccountId == null) return true;
        return !sourceAccountId.equals(destinationAccountId);
    }
}
```

> **Bean Validation Best Practice (Null Handling):** Notice how `MinimumAgeValidator` and `isNotSelfTransfer()` return `true` when a field is `null`. That is standard Jakarta Validation convention so `@NotNull` reports missing fields without duplicate misleading messages!

5. Planned Code: DTO & Controller Hardening Summary

Additional surgical updates across existing DTOs and Controllers:

Hardened Request DTOs (Monetary Precision, Auth Password Match & Customer Age) — MODIFIED

```
// 1. DepositRequest.java & WithdrawRequest.java
@NotNull(message = "Account id is required")
@Positive(message = "Account id must be positive")
private Long accountId;

@NotNull(message = "Amount is required")
@DecimalMin(value = "1.00", message = "Amount must be at least 1.00")
@Digits(integer = 16, fraction = 2, message = "Amount must have at most 2 decimal places")
private BigDecimal amount;

// 2. CreateAccountRequest.java & CreateLoanRequest.java
@Positive(message = "Customer id must be positive") private Long customerId;
@Digits(integer = 16, fraction = 2, message = "Amount must have at most 2 decimal places") private BigDecimal
openingBalance; // & principalAmount
@Digits(integer = 3, fraction = 2, message = "Interest rate format invalid")
@DecimalMax(value = "40.00", message = "Interest rate cannot exceed 40%") private BigDecimal interestRate;

// 3. CreateCustomerRequest.java & UpdateCustomerRequest.java
@NotNull(message = "Date of birth is required")
@Past(message = "Date of birth must be in the past")
@MinimumAge(value = 18, message = "Customer must be at least 18 years old for banking KYC")
private LocalDate dateOfBirth;

// 4. RegisterRequest.java / ResetPasswordRequest.java / ChangePassowordRequest.java
@AssertTrue(message = "Password and confirmPassword do not match")
public boolean isPasswordsMatching() {
    if (password == null || confirmPassword == null) return true;
    return password.equals(confirmPassword);
}
```

Controllers: PathVariable & RequestParam Validation (@Validated) — MODIFIED (All 8 Controllers)

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;
import org.springframework.validation.annotation.Validated;

@Validated
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountController {

    @GetMapping("/{accountId}")
    public ResponseEntity<AccountResponse> getAccountById(
            @PathVariable @Positive(message = "Account id must be positive") Long accountId) { ... }

    @GetMapping("/number/{accountNumber}")
    public ResponseEntity<AccountResponse> getAccountByAccountNumber(
            @PathVariable @NotBlank(message = "Account number is required") String accountNumber) { ... }
}
```

6. Verification Checklist (curl Tests)

Test Scenario	Sample Request	Expected Response
Negative Path ID	`GET /api/v1/accounts/-5`	`400` (`fieldErrors.accountId`)
Sub-paise Decimal Precision	`POST /api/v1/transactions/deposit` `{"accountId":1,"amount":10.123}`	`400` (`amount: at most 2 decimals`)
Both Transfer Targets Set	`POST /api/v1/transactions/transfer` (both `destinationAccountId` & `beneficiaryId`)	`400` (`targetExclusive` failure)
Underage / Future DOB	`POST /api/v1/customers` with `dateOfBirth: "2020-01-01"`	`400` (`MinimumAge` error)
Invalid Enum / Malformed JSON	`POST /api/v1/loans` with `"loanType": "CRYPTO"`	`400` (NOT 500!)